



Topics on Distributed Reinforcement Learning

Reinforcement learning (RL) has recently been applied to many real-world decision-making problems such as gaming, robotics, and yes, Large Language Models (LLMs). However, despite its impressive performances in simulation, RL often suffers from poor sample efficiency, which hinders its success in real-world applications. For example, when RL is applied to provide clinical decision support, its performance is limited by the number (i.e., sample size) of admission records possessed by a hospital, which cannot be synthetically generated. A natural solution is to encourage multiple RL agents (i.e., different hospitals) to share their trajectories, to collectively build a better decision-making policy that one single agent can not obtain by itself.

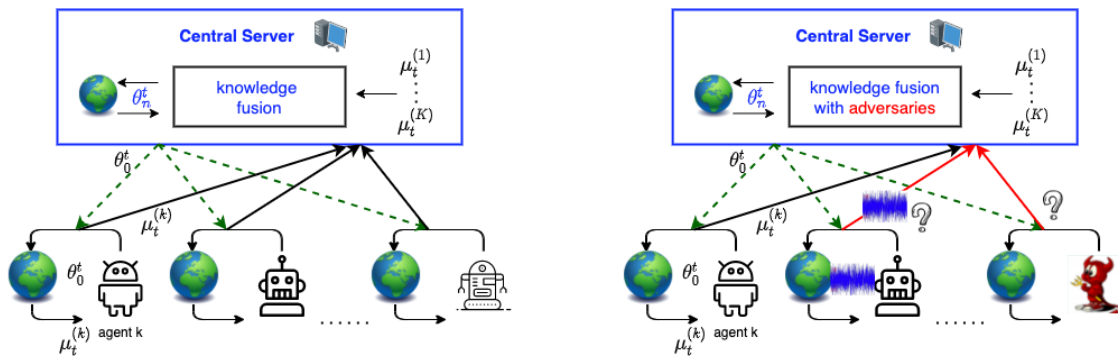


Figure 1: Illustration of our federated policy gradient framework [1]

However, in many applications, raw RL trajectories contain sensitive information (e.g., the medical records contain sensitive information about patients), and thus sharing them is prohibited. How can we collectively learn a better RL policy from distributed agents without sharing their trajectories? How do we ensure that the privacy of agents is protected? What happens if adversarial agents are present? In this project, we aim to leverage distributed computing to develop innovative techniques for distributed and decentralized reinforcement learning, and address many interesting open problems, such as privacy and robustness in RL.

Interested? Please contact us for more details!

Contact

- Flint Xiaofeng Fan: xiafan@ethz.ch, ETZ G97

References

- [1] Xiaofeng Fan, Yining Ma, Zhongxiang Dai, Wei Jing, Cheston Tan, and Bryan Kian Hsiang Low. Fault-tolerant federated reinforcement learning with theoretical guarantee. *Advances in neural information processing systems*, 34:1007–1021, 2021.

Detailed Project Topic

This project aims to immerse the student in the practical implementation of Federated Reinforcement Learning—more broadly, distributed and decentralized decision-making—by using Flower.ai (<https://flower.ai/>) as the core orchestration framework. The student will build and refine baseline FedRL algorithms (e.g., federated policy-gradient, asynchronous policy updates) within Flower.ai’s Strategy API and contribute these implementations back to the Flower GitHub repository. Building on these baselines, the student will design and implement a real-world application scenario—for example, federated traffic-signal control across multiple simulated intersections, multi-robot coordination in PettingZoo grid-worlds, distributed IoT resource allocation using MuJoCo agents, or financial risk assessment using distributed sequential data—to showcase how decentralized training yields robust, scalable intelligence. To make the project tangible and engaging, the student will develop a simple web interface (e.g., with Streamlit) that visualizes live federated metrics—per-client reward trajectories, communication round statistics, and aggregated model parameters—and deliver an interactive demo of the decentralized system during the final presentation.

Detailed Project Outline

We denote the following primary tasks mandatory (on the right side you find a rough estimate for the time that we allocate to the respective task):

- Literature research and review of federated RL fundamentals and Flower.ai documentation (★)
- Environment and baseline setup: install Flower.ai, configure PettingZoo and MuJoCo environments, implement independent RL (e.g., PPO/DQN) agents (★)
- Prototype federated policy-gradient (FedPG) demo using Flower.ai’s Strategy API with a simple MLP policy (★)
- (Optional) Implement asynchronous FedPG (AFedPG) with delay-adaptive updates in Flower.ai (★★)
- Design and implement a real-world application scenario (choose one: federated traffic-signal control, multi-robot grid-world, IoT resource allocation, or financial risk assessment) (★★)
- Develop a simple web interface (e.g., Streamlit) to visualize live federated metrics (per-client reward curves, communication statistics, aggregated parameters) (★★)
- Integrate experiment tracking and logging (Weights & Biases or TensorBoard) for reproducible evaluation (★)
- Execute empirical experiments: compare Independent Learning, FedPG, AFedPG and centralized baselines; perform ablation studies (★★★)
- Contribute baseline FedRL implementations and documentation to the Flower.ai baselines repository (★)
- Compile results, analyze findings, and write the thesis report (★★)
- Prepare and deliver the final presentation with interactive demo (★)

Extensions

Beyond the mandatory tasks, the student may explore one or more of the following extensions for deeper insights and potential publication:

- Integrate differential privacy or secure aggregation mechanisms into Flower.ai to protect client updates.
- Trigger federated updates only when local policy divergence exceeds a threshold.
- Explore peer-to-peer or hierarchical federation architectures in place of a central server.
- Design a shared replay buffer to improve sample efficiency across clients.
- Evaluate robustness when clients operate under different environment dynamics or action spaces.
- Dockerize the server, clients, and web UI for reproducible demos and easy sharing.
- Enhance the web UI with real-time controls to pause/resume clients or adjust aggregation frequency on the fly.

The Student's Duties

- One meeting per week with the advisors to discuss current matters.
- A final report in English, presenting work and results.
- A final presentation (15 min) of the work and results obtained in the project.